Product Requirements Document (PRD)

MergeLens

AI-Powered Dependency Review for GitHub

Version: 1.0

---

1. Executive Summary

MergeLens is a GitHub App that reviews dependency update pull requests created by tools like Dependabot and Renovate.

Instead of creating dependency updates, MergeLens answers the questions developers actually have after a dependency PR appears:

- Should I merge this?
- Will this break my code?
- Which files are affected?
- What APIs changed?
- How difficult is the migration?
- Can this be fixed automatically?

MergeLens combines deterministic repository analysis with AI reasoning to produce repository-specific migration intelligence.

It acts like an experienced senior engineer reviewing every dependency update before it reaches the developer.

---

2. Problem Statement

Dependency update automation has largely solved discovering new versions.

It has not solved understanding them.

Current workflow:

Dependency Update

↓

Developer reads release notes

↓

Reads migration guide

↓

Searches GitHub Issues

↓

Searches Reddit

↓

Searches StackOverflow

↓

Runs tests

↓

Fixes failures manually

↓

Eventually merges

This process consumes significant engineering time.

---

3. Pain Points

3.1 Unknown Repository Impact

Developers don't know:

- Will this update break anything?
- Which files are affected?
- Which APIs changed?

3.2 Poor Prioritization

Large repositories may receive hundreds of dependency PRs.

Developers don't know:

- Which updates are urgent.
- Which can wait.
- Which contain security fixes.
- Which have breaking changes.

3.3 Information Fragmentation

Developers manually gather information from:

- GitHub Releases
- CHANGELOG
- Migration Guides
- GitHub Issues
- Reddit
- StackOverflow
- Blog posts

3.4 Repository-Specific Analysis

Release notes are generic.

Developers need repository-specific answers.

3.5 Manual Migration

After identifying breaking changes, developers manually modify their codebase.

3.6 Lack of Confidence

Even after CI passes:

"Is this safe to merge?"

Developers lack confidence.

3.7 Community Intelligence

Developers often search:

"React 19 issue"

"Axios regression"

"Next.js latest bug"

There is no unified summary.

---

4. Vision

MergeLens becomes the intelligence layer sitting on top of existing dependency bots.

Package Registry

↓

Dependabot / Renovate

↓

MergeLens

↓

Developer

---

5. Goals

Primary Goals

- Understand dependency updates.
- Analyze repository impact.
- Estimate migration effort.
- Explain breaking changes.
- Recommend whether to merge.

Secondary Goals

- Generate migration patches.
- Validate generated patches.
- Learn repository patterns.

---

6. Non Goals (MVP)

MergeLens will NOT:

- Replace Dependabot.
- Replace Renovate.
- Become another package manager.
- Replace security scanners.
- Replace CI/CD.

---

7. Target Users

Primary

- Software Engineers
- Open Source Maintainers
- Startup Engineering Teams

Secondary

- DevOps Engineers
- Platform Engineers
- Enterprise Engineering Teams

---

8. Core Workflow

1. Dependency PR arrives.

↓

2. GitHub App receives webhook.

↓

3. Repository Knowledge Base loads.

↓

4. Release Intelligence analyzes update.

↓

5. Impact Engine maps update to repository.

↓

6. Evidence Builder prepares structured facts.

↓

7. AI generates reasoning.

↓

8. MergeLens posts review.

---

9. Functional Requirements

Repository Indexing

On installation:

- Clone repository.
- Detect language.
- Detect framework.
- Parse dependency files.
- Build AST.
- Build symbol index.
- Build API usage index.
- Build call graph.
- Detect tests.
- Store repository model.

Incremental updates only after initial indexing.

Release Intelligence

For every dependency update:

Collect:

- GitHub Release
- CHANGELOG
- Migration Guide
- Documentation
- Security Advisory

Extract:

- Breaking APIs
- Deprecated APIs
- Removed APIs
- Behavior Changes
- Performance Improvements
- Security Fixes

Repository Impact Engine

Compare:

Changed APIs

vs

Repository Usage

Produce:

- Files affected
- Symbols affected
- Functions affected
- Modules affected
- Tests affected
- Blast radius

Community Intelligence

Aggregate:

- GitHub Issues
- GitHub Discussions

Future:

- Reddit
- StackOverflow

Extract:

- Known regressions
- Common migration issues
- Adoption status
- Recommended versions

AI Reasoning

Answer:

What changed?

Why does it matter?

How difficult is migration?

Is it safe?

What should the developer do?

Patch Generation (Phase 2)

Generate:

- Updated imports
- API replacements
- Code modifications
- Test updates

Validate:

- Formatter
- Linter
- Build
- Tests

GitHub Integration

Create:

- Pull Request Comment
- GitHub Check Run
- Optional Review
- Optional Commit

---

10. Report Format

Every dependency PR produces:

Executive Summary

Priority Score

Risk Score

Breaking Changes

Affected Files

Migration Steps

Community Warnings

Security Notes

Suggested Fixes

Confidence Score

Recommendation

---

11. Repository Knowledge Base

Stores:

Repository

↓

Languages

Frameworks

Package Managers

Dependency Graph

Symbol Index

Call Graph

Import Graph

API Usage

Test Mapping

---

12. Architecture

GitHub App

↓

Webhook Service

↓

Job Queue

↓

Repository Indexer

↓

Release Intelligence

↓

Repository Impact Engine

↓

Community Intelligence

↓

Evidence Builder

↓

AI Reasoner

↓

Patch Generator

↓

Validator

↓

GitHub Review

---

13. Tech Stack

Backend

Python

FastAPI

Repository Parsing

Tree-sitter

AST

Native language parsers

Graph Processing

NetworkX

Database

PostgreSQL

Cache

Redis

Queue

FastAPI Background Tasks (MVP)

Deployment

Docker

Railway / Render

GitHub

GitHub App

AI

OpenRouter

Embeddings

Sentence Transformers

---

14. AI Philosophy

AI should only perform reasoning.

Never parsing.

Never indexing.

Never repository searching.

Deterministic systems collect evidence.

AI synthesizes evidence.

---

15. Evidence Builder

Instead of sending:

Entire repository

Entire changelog

Send:

Package

Current Version

Target Version

Breaking APIs

Repository Usage

Affected Files

Migration Guide

Community Issues

Security Notes

This minimizes token usage.

---

16. Priority Scoring

Factors

Security

Breaking Changes

Repository Usage

Community Stability

Migration Cost

Version Adoption

Recent Regression Reports

Output

0–100

---

17. Risk Score

Factors

Major Version

Removed APIs

Behavior Changes

Test Coverage

Community Reports

Generated

Low

Medium

High

Critical

---

18. Confidence Score

Factors

Static Analysis

Build Success

Test Success

AI Confidence

Community Stability

Output

0–100%

---

19. GitHub Comment Example

Executive Summary

This update introduces two breaking API changes.

One removed API is currently used in three files.

Migration is estimated to take approximately fifteen minutes.

No widespread regressions were found.

Recommendation:

Merge after replacing the deprecated API.

Confidence:

93%

---

20. API Endpoints

POST

/webhook

POST

/analyze

POST

/reindex

GET

/report

GET

/dashboard

---

21. Security

- Read-only repository permissions by default.
- Optional write permission for AI-generated commits.
- Installation scoped per repository.
- No source code retained after analysis unless explicitly enabled.

---

22. Future Roadmap

Phase 1

Repository-aware dependency analysis.

Phase 2

Automatic migration patch generation.

Phase 3

Framework upgrade support.

Examples:

React

Next.js

Angular

Vue

Spring

Django

FastAPI

.NET

Phase 4

Language upgrades.

Python

Java

Go

Rust

Node.js

Phase 5

Organization-wide dependency intelligence.

---

23. Success Metrics

Technical

- Repository indexing < 2 minutes.

- Dependency analysis < 60 seconds.
- Report generation < 15 seconds after analysis.
- High-confidence impact detection.

Product

- Percentage of analyzed PRs reviewed.
- Time saved per dependency update.
- Developer acceptance of recommendations.
- Adoption across repositories.
- Weekly active repositories.

---

24. Long-Term Vision

MergeLens evolves from a dependency review assistant into a general-purpose AI code migration platform.

Initially, it reviews dependency updates.

Eventually, it assists with:

- Framework migrations
- SDK upgrades
- Language version upgrades
- Cloud SDK changes
- Infrastructure-as-Code migrations
- Large-scale repository modernization

The core differentiator remains unchanged: repository-specific intelligence backed by deterministic analysis and AI-assisted reasoning, enabling developers to make upgrade decisions with confidence.